

MATH00041: Introduction to Programming

Functions - Importing Modules & Packages

Lecture 5

Archontis Giannakidis, PhD

Outline

- **Functions**
- **Modules**
- **Packages**

Python Functions

A function is a section of code that performs a specific task when it is called by other parts of the program. It takes in input parameters, processes them, and then returns an output. The input parameters are the values that are passed into the function when it is called, and the output is the result that is generated by the function after it has processed the input.

Creating a Function

In Python, a function is defined using the `def` keyword:

Example:

```
def my_function():  
    print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

Example:

```
def my_function():  
    print("Hello from a function")
```

```
my_function()
```

Output: Hello from a function

Arguments

Functions in programming allow for information to be passed as arguments. These arguments are placed within the parentheses of the function and can be separated by commas if there are multiple arguments. For instance, consider the following example of a function that takes one argument, `fname`. When the function is called, a first name is passed as an argument and used within the function to print the full name.

Example:

```
def my_function(fname):  
    print(fname + " Parker")
```

```
my_function("Emil")  
my_function("Tobias")  
my_function("Linus")
```

Output: Emil Parker

Output: Tobias Parker

Output: Linus Parker

Parameters or Arguments?

In programming, the terms "parameter" and "argument" are often used interchangeably to refer to information that is passed into a function. From the perspective of a function, a parameter is a variable that is listed inside the parentheses of the function definition. On the other hand, an argument is the value that is sent to the function when it is called.

Number of Arguments

By default, a function must be called with the correct number of arguments. This means that if the function requires 2 arguments, it must be called with 2 arguments, no more and no less.

Example:

his function expects 2 arguments, and gets 2 arguments:

```
def my_function(fname, lname):  
    print(fname + " " + lname)
```

```
my_function("Emil", "Parker")
```

Output: Emil Parker

- ❖ Note: If you try to call the function with 1 or 3 arguments, you will get an error

Arbitrary Arguments, *args

If you do not know how many arguments will be passed into your function, add a `*` before the parameter name in the function definition. This way, the function will receive a *tuple* of arguments and can access the items accordingly:

Example:

If the number of arguments is unknown, add a `*` before the parameter name:

```
def my_function(*kids):  
    print("The youngest child is " + kids[2])  
  
my_function("Emil", "Tobias", "Linus")
```

Output: The youngest child is Linus

Keyword Arguments

You can send arguments with the syntax `key=value`, making the order of the arguments irrelevant.

Example:

```
def my_function(child3, child2, child1):  
    print("The youngest child is " + child3)
```

```
my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

Output: The youngest child is Linus

Arbitrary Keyword Arguments, **kwargs

If you do not know how many keyword arguments that will be passed into your function, add two asterisks: ****** before the parameter name in the function definition.

This function receives a dictionary of arguments for item access.

Example:

If the number of keyword arguments is unknown, add a double ****** before the parameter name:

```
def my_function(**kid):  
    print("His last name is " + kid["lname"])  
  
my_function(fname = "Tobias", lname = "Parker")
```

Output: His last name is Parker

Default Parameter Value

In Python, you can define a function with default parameter values. When the function is called, it checks if any argument is passed to it. If an argument is passed, it uses the passed value. If no argument is passed, it uses the default value specified in the function definition.

Example:

```
def my_function(country = "Norway"):  
    print("I am from " + country)
```

```
my_function("Sweden")
```

```
my_function("India")
```

```
my_function()
```

```
my_function("Brazil")
```

Output: I am from Sweden

Output: I am from India

Output: I am from Norway

Output: I am from Brazil

Passing a List as an Argument

It is possible to send different data types of arguments (string, number, list, dictionary, etc.) to a function, and they will be treated as the same data type inside the function. For instance, if you pass a List as an argument, it will remain a List when it reaches the function:

Example:

```
def my_function(food):  
    for x in food:  
        print(x)
```

```
fruits = ["apple", "banana", "cherry"]
```

```
my_function(fruits)
```

Output: apple

Output: banana

Output: cherry

Return Values

The statement **"return"** is used in a function to send back a value as the result of the function. Without the return statement, the function would execute but it wouldn't return anything. By returning a value, the function can be used in other parts of the program to perform further operations.

Example:

```
def my_function(x):  
    return 5 * x  
  
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```

Output: 15

Output: 25

Output: 45

The pass Statement

In Python, function definitions are used to define a block of code that can be used repeatedly. It is important to note that function definitions must contain code, otherwise, you will get an error. However, if you have an empty function definition for some reason, you can avoid getting an error by including a 'pass' statement. The 'pass' statement is a null operation, meaning it does nothing, but it can be used as a placeholder to avoid syntax errors.

Example:

```
def myfunction():  
    pass
```

Output:

What is a Module?

A module is a collection of code that can be used in other programs. It typically contains a set of related functions that can be imported and used in a larger application. Think of it like a toolbox that you can use to build your application. By importing a module, you can take advantage of the pre-existing code and avoid duplicating efforts. This can save time and improve the quality of your application.

Create a Module

A module is a file containing Python code that can be imported and used in other Python programs. To create a module, you just need to save your code in a file with the `.py` extension. Once you've saved your code in a `.py` file, you can import it into another Python program using the 'import' statement, and then use the functions, classes, or variables defined in the module in your program. This allows you to reuse code and make your programs more modular and organized.

Example:

Save this code in a file named `mymodule.py`

```
def greeting(name):  
    print("Hello, " + name)
```

Use a Module

When we create a module in Python, we can use the functionality of it in other programs by importing it into the code. This can be done by using the **'import'** statement followed by the name of the module we created. Once the module is imported, its functionality can be used in the code to perform specific tasks. In the given text, it is explained that after creating the module, we can use it in other programs by importing it using the **'import'** statement.

Example:

Import the module named mymodule, and call the greeting function:

```
import mymodule
```

```
mymodule.greeting("Jonathan")
```

Output: Hello, Jonathan

❖ Note: When using a function from a module, use the syntax: module_name.function_name.

Variables in Module

In programming, a module is essentially a file that contains code. It can contain functions, which are blocks of code that perform a specific task and can be called from other parts of the code. However, a module can also have variables of many different types, such as arrays, dictionaries, objects, and more. These variables can hold data or other information that the code needs to perform its tasks. By including both functions and variables, modules can provide a way to organize and reuse code across different parts of a program.

Example:

Save this code in the file `mymodule.py`

```
person1 = {  
    "name": "John",  
    "age": 36,  
    "country": "Norway"  
}
```

Example:

Import the module named mymodule, and access the person1 dictionary:

```
import mymodule  
  
a = mymodule.person1["age"]  
print(a)
```

Output: 36

Naming a Module

When you write a program in Python, you need to save it in a file with a ".py" extension. You can choose any name for this file, but it's important to make sure that it ends with ".py".

Re-naming a Module

When you import a module in Python, you can give it an alternative, shorter name by using the 'as' keyword. This is called creating an alias. The purpose of creating an alias is to make the code more readable and easier to type.

Example:

Create an alias for `mymodule` called `mx`:

```
import mymodule as mx
```

```
a = mx.person1["age"]
```

```
print(a)
```

Output: 36

Built-in Modules

Python is a programming language that comes with a variety of pre-built modules that you can use to perform specific tasks. These modules can be imported into your code whenever you need them. By using these built-in modules, you can save time and effort while coding and make your programs more efficient.

Example:

Import and use the `platform` module:

```
import platform
```

```
x = platform.system()
```

```
print(x)
```

Output: Windows

Using the dir() Function

The `dir()` function is a built-in Python function that can be used to list all the function and variable names that are defined within a module. It can be quite useful when working with large or complex modules, as it allows you to quickly see all the available functions and variables that you can use.

Example:

List all the defined names belonging to the platform module:

```
import platform
```

```
x = dir(platform)
print(x)
```

```
Output: ['_WIN32_CLIENT_RELEASES', '_WIN32_SERVER_RELEASES', '__builtins__', '__cached__',
'__copyright__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__',
'__version__', '_comparable_version', '_component_re', '_default_architecture',
'_follow_symlinks', '_ironpython26_sys_version_parser', '_ironpython_sys_version_parser',
'_java_getprop', '_libc_search', '_mac_ver_xml', '_node', '_norm_version', '_platform',
'_platform_cache', '_pypy_sys_version_parser', '_sys_version', '_sys_version_cache',
'_sys_version_parser', '_syscmd_file', '_syscmd_uname', '_syscmd_ver', '_uname_cache',
'_ver_output', '_ver_stages', 'architecture', 'collections', 'java_ver', 'libc_ver', 'mac_ver', 'machine',
'node', 'os', 'platform', 'processor', 'python_branch', 'python_build', 'python_compiler',
'python_implementation', 'python_revision', 'python_version', 'python_version_tuple', 're',
'release', 'sys', 'system', 'system_alias', 'uname', 'uname_result', 'version', 'win32_edition',
'win32_is_iot', 'win32_ver']
```

Import From Module

In Python, a module is a file containing Python definitions and statements. Sometimes, you may only need to use a specific part of a module. In such cases, you can use the 'from' keyword to import only the necessary parts of the module, rather than importing the entire module. This can help reduce clutter in your code and make it easier to read and understand.

Example:

The module named `mymodule` has one function and one dictionary:

```
def greeting(name):  
    print("Hello, " + name)  
  
person1 = {  
    "name": "John",  
    "age": 36,  
    "country": "Norway"  
}
```

Example:

Import only the `person1` dictionary from the module:

```
from mymodule import person1  
  
print (person1["age"])
```

Output: 36

- ❖ Note: When importing using the `from` keyword, do not use the module name when referring to elements in the module. Example: `person1["age"]`, not `mymodule.person1["age"]`

What is a Package?

Python is a programming language that allows you to organize your code into separate modules, which can be used across different projects. A module can contain functions, classes, and variables that are relevant to a specific task. A package is a folder that contains one or more modules, along with any other files that may be necessary, such as documentation or configuration files. So, when you want to use a module in your project, you can simply import it from the package and use its functions and classes.

What is PIP?

The original text explains what PIP is, but it could be improved by stating more clearly that PIP is a package manager that helps with the installation and management of Python packages or modules. The rewritten text makes it more concise and clear by stating that PIP is a package manager for Python, and it elaborates on what the package manager does by expanding on how it can be used to install and manage Python packages or modules.

❖ Note: If you have Python version 3.4 or later, PIP is included by default.

Check if PIP is Installed

Navigate your command line to the location of Python's script directory, and type the following:

Example:

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>pip --version
```

Install PIP

If you do not have PIP installed, you can download and install it from this page:

<https://pypi.org/project/pip/>

Download a Package

To download a package, follow these steps:

1. Open the command line interface.
2. Use PIP to download the package you want.
3. Navigate to Python's script directory.
4. Enter the required command to download the package.

Example:

Download a package named "camelcase":

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>pip install camelcase
```

Now you have downloaded and installed your first package!

Using a Package

Once the package is installed, it is ready to use.

Import the "camelcase" package into your project.

Example:

Import and use "camelcase":

```
import camelcase

c = camelcase.CamelCase()

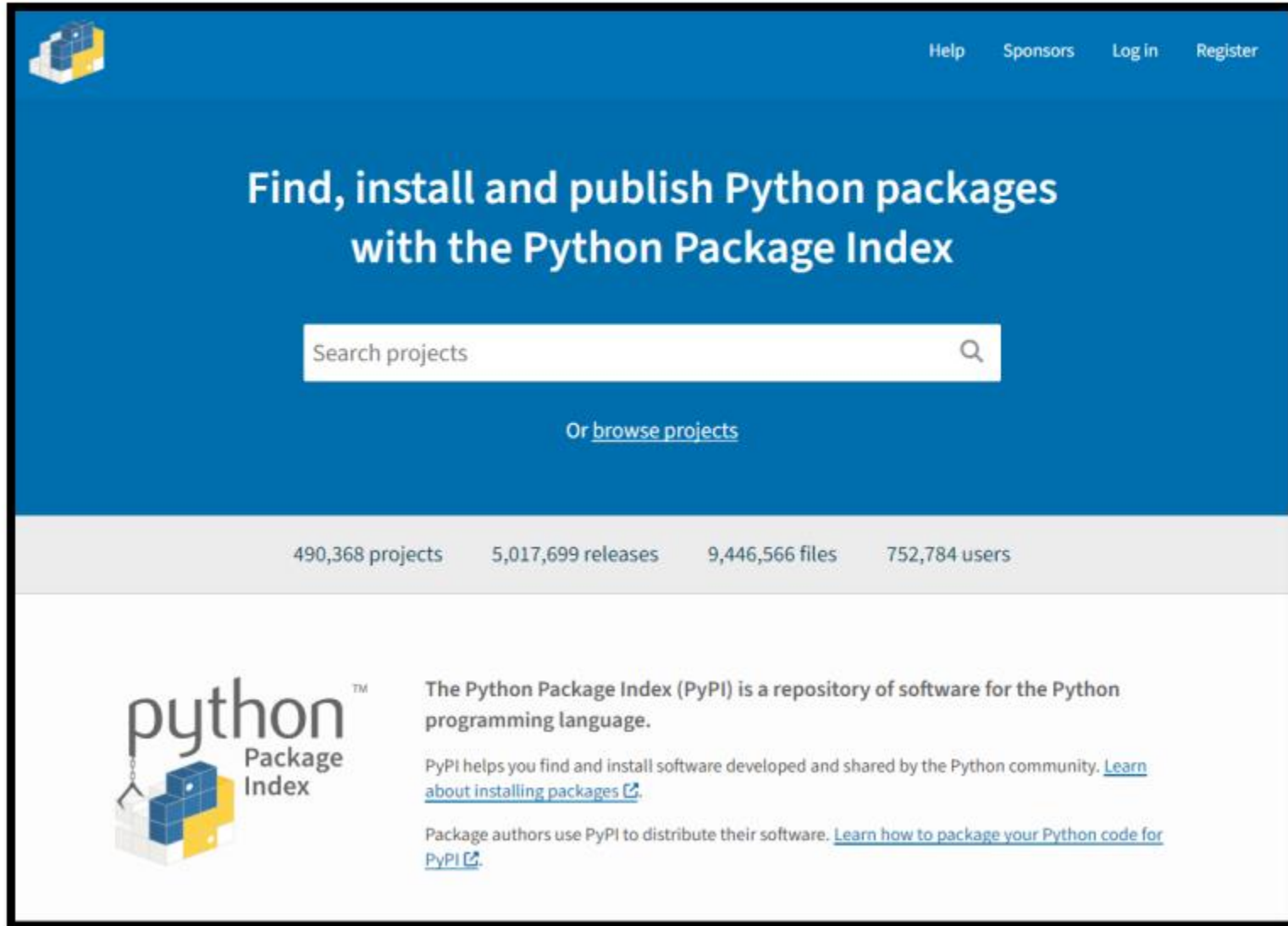
txt = "hello world"

print(c.hump(txt))
```

Output: Lorem Ipsum Dolor Sit Amet #This method capitalizes the first letter of each word.

Find Packages

You can find additional packages by visiting <https://pypi.org/>



The screenshot shows the PyPI homepage with a blue header. In the top left is the PyPI logo (a stack of blue and yellow cubes). In the top right are links for Help, Sponsors, Log in, and Register. The main heading reads "Find, install and publish Python packages with the Python Package Index". Below this is a search bar with the placeholder text "Search projects" and a magnifying glass icon. Under the search bar is a link "Or [browse projects](#)". A light gray bar contains statistics: "490,368 projects", "5,017,699 releases", "9,446,566 files", and "752,784 users". The footer features the "python Package Index" logo on the left and descriptive text on the right. The text states that PyPI is a repository for Python software, helps find and install community-developed software, and is used by authors to distribute their software. It includes two links: "Learn about installing packages" and "Learn how to package your Python code for PyPI".

Help Sponsors Log in Register

Find, install and publish Python packages with the Python Package Index

Search projects

Or [browse projects](#)

490,368 projects 5,017,699 releases 9,446,566 files 752,784 users

python
Package Index

The Python Package Index (PyPI) is a repository of software for the Python programming language.

PyPI helps you find and install software developed and shared by the Python community. [Learn about installing packages](#)

Package authors use PyPI to distribute their software. [Learn how to package your Python code for PyPI](#)

Remove a Package

If you're using pip, you can uninstall a package using the following command:

```
``pip uninstall package_name``
```

Simply replace `package\_name` with the name of the package you want to remove.

Example:

Uninstall the package named "camelcase":

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>pip uninstall camelcase
```

The PIP Package Manager will ask you to confirm that you want to remove the camelcase package:

```
Uninstalling camelcase-02.1:
  Would remove:
    c:\users\Your Name\appdata\local\programs\python\python36-32\lib\site-packages\camelcase-0.2-py3.6.egg-info
    c:\users\Your Name\appdata\local\programs\python\python36-32\lib\site-packages\camelcase\*
  Proceed (y/n)?
```

Press **y** and the package will be removed.

List Packages

To view all the packages installed on your system, you can use the 'list' command.

Example:

List installed packages:

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>pip list
```

Result:

Package	Version

camelcase	0.2
mysql-connector	2.1.6
pip	18.1
pymongo	3.6.1
setuptools	39.0.1

Important Packages

1. **NumPy**: A fundamental package for numerical computations in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.
2. **Pandas**: A powerful data analysis and manipulation library. It provides data structures like Series and DataFrames, making it easy to handle and analyze structured data.
3. **Matplotlib**: A popular plotting library for creating static, animated, and interactive visualizations in Python.
4. **Requests**: A simple HTTP library for Python, used to send HTTP requests easily.
5. **Flask**: A micro web framework for Python. It's widely used for web development, especially for beginners due to its simplicity.

6. **Beautiful Soup**: A library for pulling data out of HTML and XML files. It's commonly used for web scraping.
7. **Scikit-learn**: A machine learning library that provides simple and efficient tools for data mining and data analysis. It's built on NumPy, SciPy, and matplotlib.
8. **SciPy**: An ecosystem of open-source software for mathematics, science, and engineering. It's built on the NumPy extension and provides many user-friendly and efficient numerical routines.
9. **TensorFlow**: An end-to-end open-source platform for machine learning. It's a bit more advanced but is often used by beginners interested in diving into neural networks and deep learning.
10. **PyGame**: A set of Python modules designed for writing video games. It's a great way for beginners to learn Python while having fun creating simple games.

11. **Jupyter Notebook:** Not a library per se, but an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text.
12. **Selenium:** A tool for automating web browsers. It's used for browser-based automation for testing or web scraping.
13. **Pillow (PIL Fork):** A Python Imaging Library that adds support for opening, manipulating, and saving many different image file formats.
14. **Django:** A high-level Python Web framework that encourages rapid development and clean, pragmatic design. It's a bit more complex but commonly used by beginners interested in web development.
15. **SQLAlchemy:** A SQL toolkit and Object-Relational Mapping (ORM) library for Python. It provides a full suite of well-known enterprise-level persistence patterns.

These packages provide a solid foundation for various types of projects and are great tools for beginners to get familiar with.

Thank you